Homework 2

Due: 11:59pm on March 27, 2025 (Thursday)

Objectives:

- Understand how Queue/Deque can be used as an aid in an algorithm.
- Explore and compare different data structures (ArrayDeque and LinkedList) and their methods to solve the same problem.

Josephus Problem (from Wikipedia)
There are people standing in a circle waiting to be executed. A certain number of people are skipped and then one person is executed. This is repeated starting with the person after the person who was executed. The elimination proceeds around the circle (which is becoming smaller as the executed people are removed) until one person remains.
Named after Flavius Josephus, a Jewish historian living in the 1st century. According to Josephus' account of the siege of Yodfat, he and his comrade soldiers were trapped in a cave, the exit of which was blocked by Romans. They chose death over capture and decided that they would form a circle and start eliminating themselves using a step of three. Josephus and another man remained alive last and gave up to the Romans.

The goal is to determine where to stand in the circle so that you are the last person alive.

Imagine that you are in the same situation just like Flavius Josephus. You are in danger!

However, you are a programmer and smart enough to build an algorithm that tells you where to stand in the circle to survive.

As you can imagine, time is critical here. You need to implement and compare three different methods in the `Josephus` class and decide which method is the fastest to use for you to survive.

You are to implement three different methods to solve the Josephus problem.

Step-by-step guide:

- Understand how Josephus problem works.
    - For your understanding, the steps of execution are as follows using Queue data structure (size = 6, rotation = 3).

      Initial Circle: 1 2 3 4 5 6
      After the 1st execution: 4 5 6 1 2
      After the 2nd execution: 1 2 4 5
      After the 3rd execution: 5 1 2
      After the 4th execution: 5 1
      After the 5th execution: 1 (1 survives)

- Read carefully the JavaDoc comments of each method in the `Josephus` class provided.
    - Do not use Node nested class or implement your own LinkedList for this assignment. In all three methods of Josephus class, you must use the following two Java classes. Think carefully about which methods of the two Java classes ought to be used.
        - ArrayDeque (https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/ArrayDeque.html)
        - LinkedList (https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/LinkedList.html)

- Implement the methods on your choice of IDE.
- Test your code extensively!
    - Test code in `HW2Driver.java` file is just a starting point to test your program. When you run the program, you should see the following result. Do not make any assumptions and make sure to print the same values as you see below.

size (number of people)	rotation	survivor
1	5	1
5	1	5
5	2	3
11	1	11
40	7	24
55	1	55
66	100	7
1000	1000	609
-1	2	throw RuntimeException instead
5	0	throw RuntimeException instead

    - Notice that rotation value can be bigger than size (i.e., the number of the people in the circle).
    - Remember! Time is critical here! You cannot afford to be slow to find out where to stand. ***So, in EACH iteration, your algorithm should be able to find the elimination position without rotating the circle more than its current size.***
    - Don't be confused: position here does not mean an index value. It means the position in the circle to survive.

Deliverables:

- Your source code file.
    - Include your Andrew ID and name in the header comments as author tag.
    - ***In the header comments, clearly write WHICH method(s) you would use out of three methods (playWithAD, playWithLL, and playWithLLAt), assuming that there are many people in the circle, to find the survivor's position and WHY.***
    - Do not use packages. This means that you should not have code like "`package hw2;`" in your `Josephus.java` file. You are allowed to import in this assignment.
    - Submit your source code file, `Josephus.java`, using Autolab (https://autolab.andrew.cmu.edu). Make sure to submit your source code file (.java file), not .class file.

Grading:

Autolab will grade your assignment as follows.

- Working code: 90 points
- Coding conventions: 10 points
    - We will deduct one point for each coding convention issue detected.

In case you do not pass test case(s), please spend some time to think about edge cases you may have missed and test thoroughly before asking questions.

Autolab will check your coding conventions quite strictly. Make sure to read the Java Coding Conventions document when in doubt.

Autolab will show you the results of its grading within approximately a few minutes of your submission. You may submit multiple times to correct any problems with your assignment. Autolab uses the last submission as your grade.

The Autolab score is not final. The TAs will look at your source code for correctness and design, and deduct points if applicable. The most important criterion is always correctness. It is also critical that your code is efficient and follows the specification properly. Additionally, it should be readable and well organized. This includes proper use of clear comments. Points will be deducted for poor design decisions and unreadable code, etc.

As mentioned in the syllabus, **late submissions will NOT be accepted**. If you have multiple versions of your code file, make sure you submit the correct version.